Lesson notes · for you

Attacking and hardening models

AI security · University · Lesson 3

Time	Materials
20 min	Paper and a pen for the table; A model you built, or the Labs app; A phone or computer for the code steps, if you have one

Learning goals

- Explain evasion, extraction and membership inference.
- Compute the worst small nudge for a linear model.
- Choose defences and measure what they cost.

Activity: Nudge the score by hand

1. Warm-up (3 min): Ask: a model is 99% sure; can a tiny change flip it? Collect guesses and reasons.
2. Score (6 min): By hand, compute the score 2 × 1 + (-1) × 1 + 1 × 1 - 1 = 1. It is positive, so the answer is “yes”.
3. Nudge (6 min): Move each input 0.5 against its weight's sign: the inputs become 0.5, 1.5 and 0.5. The new score is -1: the answer flips.
4. Certify (5 min): The drop is ε × (2 + 1 + 1) = 4ε. The answer is safe while 4ε is less than 1, so for any ε below 0.25. Compare with robust() in the code step.

Work on paper: the numbers are small on purpose.

Discussion

- Why doesn't a confident answer mean a robust one?
- Which defence would you pay for, and what would it cost?
- Why do exact scores help an attacker?

Check yourself

- Weights 3 and -2, inputs 1 and 1, bias 0, ε = 0.5: write the worst nudge and the new score.

Try it this week

Test a simple model you've built, or the lab's, twice: on clean inputs and on slightly nudged ones. Write down both accuracies, then add one defence and measure what it costs.

Worksheet

Attacking and hardening models

AI security · University · Lesson 3

Name _____ Class _____ Date _____

1 **Your model's API returns exact confidence scores to anyone, with no limits. Which attacks does that make easiest?**

- ◯ Extraction and membership inference
- ◯ Poisoning its training data
- ◯ A fake copy in the supply chain

2 **score(w, b, x) is a linear model. Finish nudge(w, x, eps) so each input moves eps against the sign of its weight: the small change that lowers the score most. The starter moves the wrong way.**

```
def score(w, b, x):
    return sum(a * c for a, c in zip(w, x)) + b

def nudge(w, x, eps):
    sign = lambda a: (a > 0) - (a < 0)
    return [c + eps * sign(a)
            for a, c in zip(w, x)]

w, b, x = [2, -1, 1], -1, [1, 1, 1]
print(score(w, b, x))
print(score(w, b, nudge(w, x, 0.5)))
```

Write your program:

3 **Complete robust(): no nudge of size eps can flip the answer when the score is further from zero than the biggest possible drop.**

```
def robust(w, b, x, eps):
    drop = eps * _____(abs(a) for a in w)
    return abs(score(w, b, x)) _____ drop
```

Word bank: `>` `sum` `max` `<`

4 **Match each attack on a model to a defence that targets what it needs.**

Evasion with adversarial examples	• • • •	Privacy noise during training
Extraction by mass querying	• • • •	Rate limits and labels only
Membership inference	• • • •	Adversarial training, and a person checks

5 **Adversarial training makes a model impossible to fool.**

◯ True ◯ False

✎ **Think of a model whose answer matters to people you know: an exam marker, a loan app, a crop app. Which of the three attacks worries you most for it, and which defence would you pay for?**